

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI

Semester: 5

Academic Year: 2026-27

Division: E

Batch: 1

Practical No: 4

Roll no: 52

PRN No: 12410371

Name of Student: Vaidik Samarth

Subject Name: Deep Learning

Title of Practical: Develop an LSTM-based model for time-series forecasting using a stock price, weather, or sales dataset.

Methodology:

1. Load the Jena Climate CSV dataset.
2. Parse and sort the date-time column.
3. Select relevant weather features.
4. Resample the 10-minute observations to hourly frequency.
5. Split the data chronologically into 80% training and 20% testing sets.
6. Fit 'StandardScaler' only on the training data.
7. Create 24-hour sliding windows.
8. Train an LSTM regression model.
9. Evaluate predictions using MAE and RMSE.
10. Visualize training loss and actual-vs-predicted temperatures.

Code & Output:

```
import os
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf          Chat (CTRL + I) / Share (CTRL + I)
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

SEED = 42
np.random.seed(SEED)
random.seed(SEED)
tf.random.set_seed(SEED)

print("TensorFlow version:", tf.__version__)

TensorFlow version: 2.20.0
```

```
DATA_PATH = "jena_climate_2009_2016.csv"

df = pd.read_csv(DATA_PATH)

df["Date Time"] = pd.to_datetime(
    df["Date Time"],
    format="%d.%m.%Y %H:%M:%S"
)

df = df.sort_values("Date Time").reset_index(drop=True)

print("Dataset shape:", df.shape)
print("Start:", df["Date Time"].min())
print("End:", df["Date Time"].max())
print("Missing values:", df.isna().sum().sum())

display(df.head())
```

Python

	Date Time	p (mbar)	T (degC)	Tpot (K)	Tdew (degC)	rh (%)	VPmax (mbar)	VPact (mbar)	VPdef (mbar)	sh (g/kg)	H2OC (mmol/mol)	rho (g/m ³)	wv (m/s)	max_wv (m/s)	wd (deg)
0	2009-01-01 00:10:00	996.52	-8.02	265.40	-8.90	93.3	3.33	3.11	0.22	1.94	3.12	1307.75	1.03	1.75	152.3
1	2009-01-01 00:20:00	996.57	-8.41	265.01	-9.28	93.4	3.23	3.02	0.21	1.89	3.03	1309.80	0.72	1.50	136.1
2	2009-01-01 00:30:00	996.53	-8.51	264.91	-9.31	93.9	3.21	3.01	0.20	1.88	3.02	1310.24	0.19	0.63	171.6
3	2009-01-01 00:40:00	996.51	-8.31	265.12	-9.07	94.2	3.26	3.07	0.19	1.92	3.08	1309.19	0.34	0.50	198.0
4	2009-01-01 00:50:00	996.51	-8.27	265.15	-9.04	94.1	3.27	3.08	0.19	1.92	3.09	1309.00	0.32	0.63	214.3

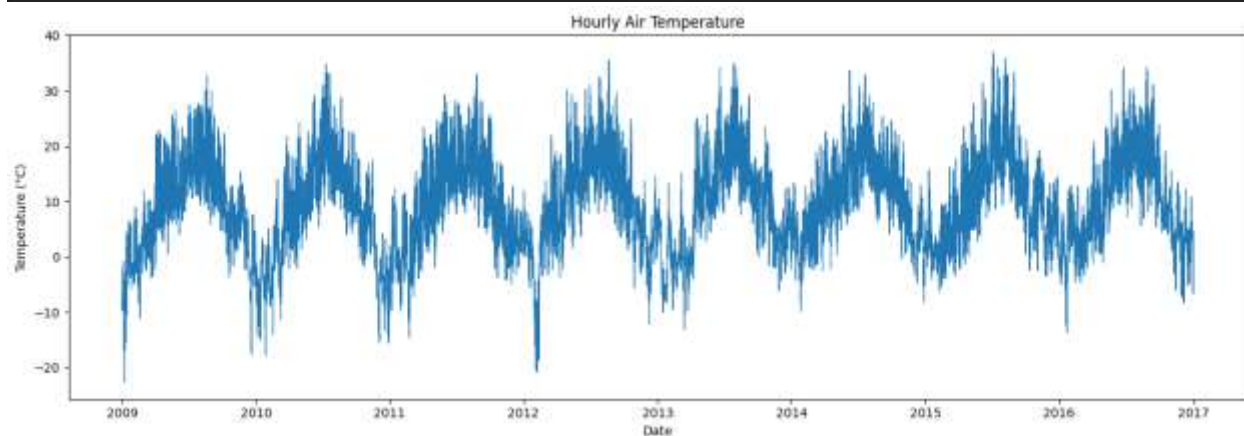
```
# Relevant variables for a compact multivariate forecasting model.
FEATURES = ["T (degC)", "p (mbar)", "rh (%)", "wv (m/s)"]
TARGET = "T (degC)"

weather = df[["Date Time"] + FEATURES].copy()

# Aggregate the 10-minute observations into hourly averages.
hourly = (
    weather.set_index("Date Time")
    .resample("1h")
    .mean()
    .dropna()
    .reset_index()
)

print("Hourly dataset shape:", hourly.shape)
display(hourly.head())
```

Python



```
split_idx = int(len(hourly) * 0.80)

train_df = hourly.iloc[:split_idx].copy()
test_df = hourly.iloc[split_idx:].copy()

print("training rows:", len(train_df))
print("testing rows :", len(test_df))
print("training period:", train_df["Date Time"].min(), "to", train_df["Date Time"].max())
print("testing period :", test_df["Date Time"].min(), "to", test_df["Date Time"].max())
```

Python

```
feature_scaler = StandardScaler()

train_values = train_df[FEATURES].values
test_values = test_df[FEATURES].values

train_scaled = feature_scaler.fit_transform(train_values)
test_scaled = feature_scaler.transform(test_values)

print("Training scaled shape:", train_scaled.shape)
print("Testing scaled shape :", test_scaled.shape)
```

Python

```

SEQUENCE_LENGTH = 24 # previous 24 hours -> predict the next hour

target_index = FEATURES.index(TARGET)

def create_sequences(data, sequence_length, target_index):
    X, y = [], []
    for i in range(sequence_length, len(data)):
        X.append(data[i-sequence_length:i])
        y.append(data[i, target_index])
    return np.array(X), np.array(y)

X_train, y_train = create_sequences(
    train_scaled, SEQUENCE_LENGTH, target_index
)

# Include the final 24 training observations as context for the first test prediction.
test_context = np.vstack([train_scaled[-SEQUENCE_LENGTH:], test_scaled])

X_test, y_test = create_sequences(
    test_context, SEQUENCE_LENGTH, target_index
)

print("X_train:", X_train.shape)
print("y_train:", y_train.shape)
print("X_test :", X_test.shape)
print("y_test :", y_test.shape)

```

```

model = tf.keras.Sequential([
    tf.keras.Input(shape=(SEQUENCE_LENGTH, len(FEATURES))),
    tf.keras.layers.LSTM(64),
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dense(1)
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="mse",
    metrics=["mae"]
)

model.summary()

```

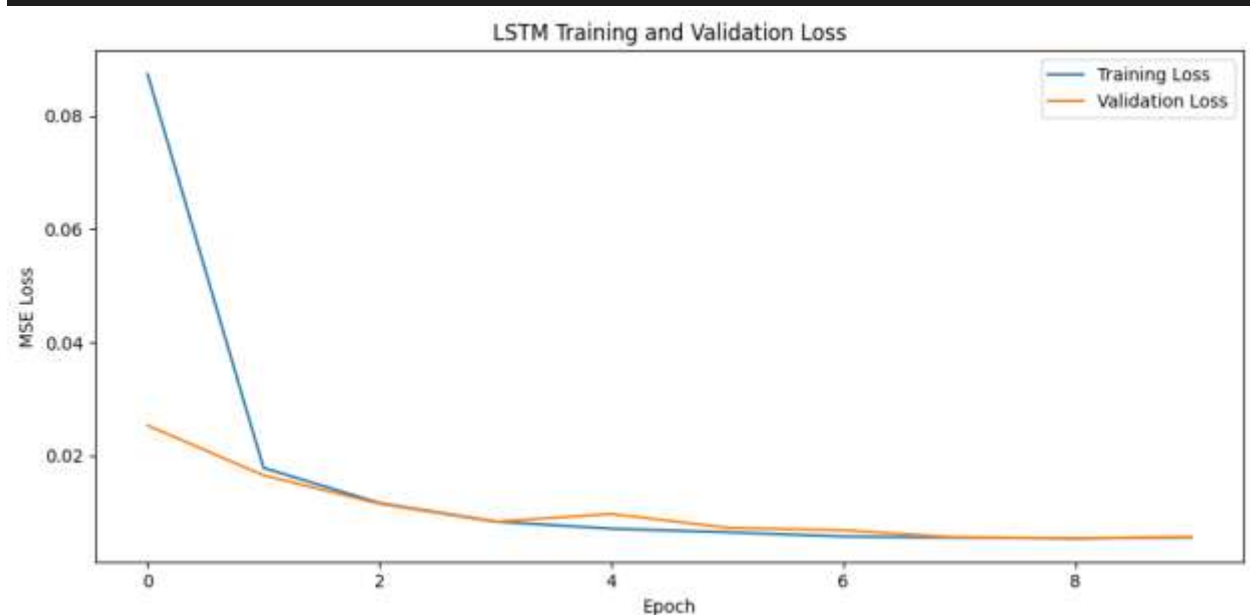
...
Model: "sequential"
...

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 64)	17,664
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

...
Total params: 19,777 (77.25 KB)
...
Trainable params: 19,777 (77.25 KB)
...
Non-trainable params: 0 (0.00 B)

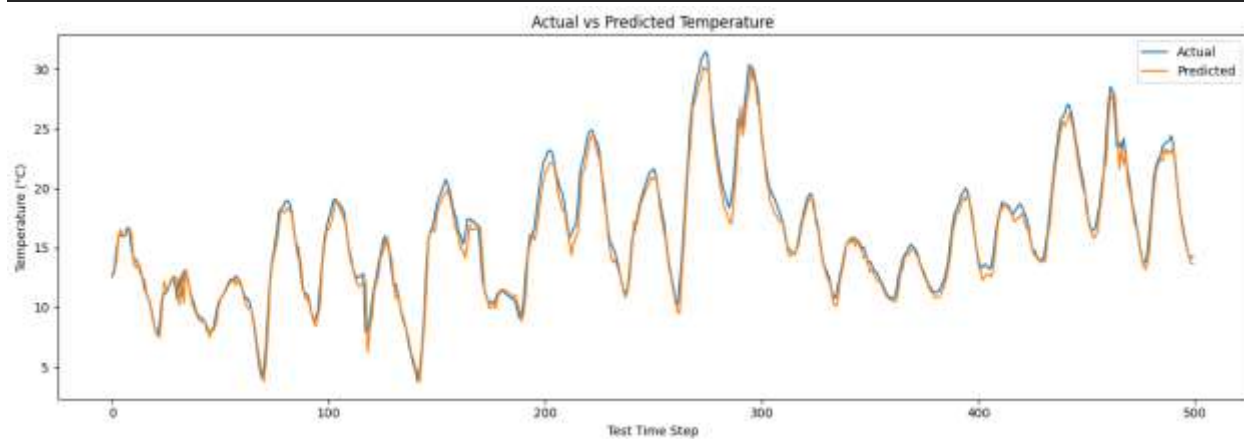
Model architecture

Input (24 × 4) → LSTM (64 units) → Dense (32, ReLU) → Dense (1)



```
# Plot a representative portion of the test period for readability.  
plot_points = min(500, len(y_test_actual))
```

```
plt.figure(figsize=(14, 5))  
plt.plot(y_test_actual[:plot_points], label="Actual")  
plt.plot(y_pred_actual[:plot_points], label="Predicted")  
plt.title("Actual vs Predicted Temperature")  
plt.xlabel("Test Time Step")  
plt.ylabel("Temperature (°C)")  
plt.legend()  
plt.tight_layout()  
plt.show()
```



```
results = pd.DataFrame({
    "Actual Temperature (°C)": y_test_actual[:10],
    "Predicted Temperature (°C)": y_pred_actual[:10]
})

display(results.round(3))
```

	Actual Temperature (°C)	Predicted Temperature (°C)
0	12.605	12.768
1	13.210	12.891
2	14.907	13.958
3	15.942	16.078
4	16.082	16.530
5	15.932	15.918
6	16.065	15.990
7	16.710	16.295
8	16.483	16.681
9	14.832	16.038

Conclusion:

The LSTM model uses the temporal relationship among recent weather observations to forecast the next-hour air temperature. Performance is evaluated using MAE and RMSE, while the actual-versus-predicted plot provides a visual check of forecasting behavior.